

Animal Deterrence Using Computer Vision and Raspberry PI

A
Project Report Submitted
In partial fulfillment of the requirements for the award of the Degree of

BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE & ENGINEERING

By

Kundeti Vamsi

20761A0597

Parsa Pallavi

20761A05B2

Gundepalli Dedeepya

20761A0585

Under the esteemed guidance of

Dr. Ch. Venkata Narayana

Professor



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
LAKIREDDY BALIREDDY COLLEGE OF ENGINEERING
(AUTONOMOUS)

Accredited by NAAC with 'A' Grade & NBA (Under Tier - I),

An ISO 21001:2018, 14001:2015, 50001:2018 Certified Institution

Approved by AICTE, New Delhi and Affiliated to JNTU, Kakinada

L.B. REDDY NAGAR, MYLAVARAM, NTR DIST., A.P.-521 230.

2020 – 2024

LAKIREDDY BALI REDDY COLLEGE OF ENGINEERING (AUTONOMOUS)

Accredited by NAAC with 'A' Grade & NBA (Under Tier - I),
A ISO 21001:2018, 14001:2015, 50001:2018 Certified Institution
Approved by AICTE, New Delhi and Affiliated to JNTUK, Kakinada
L.B. REDDY NAGAR, MYLAVARAM, NTR DIST, A.P - 521 230.

Department Of COMPUTER SCIENCE & ENGINEERING



CERTIFICATE

This is to certify that the project entitled “**Animal Deterrence Using Computer Vision And Raspberry Pi**” is being submitted by

Kundeti Vamsi
Parsa Pallavi
Gundepalli Dedeepya

20761A0597
20761A05B2
20761A0585

in partial fulfillment of the requirements for the award of degree of **B. Tech in Computer Science & Engineering** from **Jawaharlal Nehru Technological University Kakinada** is a record of bonafide work carried out by them at **Lakireddy Bali Reddy College of Engineering.**

The results embodied in this Project report has not been submitted to any other University or Institute for the award of any degree or diploma.

PROJECT GUIDE
Dr. Ch. Venkata Narayana

HEAD OF THE DEPARTMENT
Dr. D. Veeraiah

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We take great pleasure to express our deep sense of gratitude to our project guide

Dr. Ch. Venkata Narayana, Professor, for his valuable guidance during the course of our project work.

We would like to thank **Dr. D. Veeraiah**, Professor & Head of the Department of Computer Science & Engineering for his encouragement.

We would like to express our heart-felt thanks to **Dr. K. Appa Rao**, Principal, Lakireddy Bali Reddy College of Engineering for providing all the facilities for our project.

Our sincere gratitude to **Dr. B. Siva Rama Krishna**, Associate Professor, to his contribution and support throughout our project work. Our utmost thanks to all the faculty members and Non-Teaching Staff of the Department of Computer Science & Engineering for their support throughout our project work.

Our Family Members and Friends receive our deepest gratitude and love for their support throughout our academic year.

Kundeti Vamsi(20761A0597)

Parsa Pallavi(20761A05B2)

Gundepalli Dedeepya(20761A0585)

DECLARATION

We are here by declaring that the project entitled “**ANIMAL DETERRENCE USING COMPUTER VISION AND RASBERRY PI**” is a work done by us. We certify that the work contained in the report is original and has been done by us under the guidance of supervisor Dr. Ch. Venkata Narayana. The work has not been submitted to any other institute in preparing for any degree or diploma. We have followed the guidelines provided by the institute in preparing the report. We have conformed to the norms and guidelines given in the Ethical Code of Conduct of the institute. Whenever we have used materials (data, theoretical analysis, figures and text) from other sources, we have given due credit to them by citing them in the text of the report and giving their details in the references. Further, we have taken permission from the copyright’s owner of the sources, whenever necessary.

Signature(s) of the students(s)

Kundeti Vamsi (20761A0597)

Parsa Pallavi (20761A05B2)

Gundepalli Dedeepya (20761A0585)

ABSTRACT

This project introduces a sophisticated animal deterrence system, employing the YOLOv8 model and the Ultralytics framework. The system, designed to thwart unauthorized animal invasions in restricted areas, integrates cutting-edge computer vision algorithms with the computational capabilities of Raspberry Pi. In real-time, strategically positioned cameras capture video feeds, which are meticulously analyzed using YOLOv8 for precise animal identification and categorization. Upon detecting unauthorized animal presence, the system activates deterrent devices, such as alarms or lights, ensuring swift and effective response. The project's success pivots on the refinement of computer vision models and seamless Raspberry Pi-to-camera connectivity. Beyond its technical intricacies, the implications of this innovative system are vast, ranging from safeguarding agricultural yields to the preservation of wildlife habitats and the maintenance of urban green spaces. By fostering coexistence and mitigating human-animal conflicts, this project stands as a beacon of innovation in addressing contemporary challenges. The integration of Ultralytics YOLOv8 and tools like Roboflow reflects a strategic and forward-thinking approach in tackling complex real-world issues with efficiency and precision.

LIST OF CONTENTS

CONTENTS	PAGE NO
1. INTRODUCTION	1
1.1. Overview of the Project	1
1.2. Feasibility Study	
1.3. Scope	2
2. LITERATURE SURVEY	3
2.1. Existing System	5
2.2. Proposed System	6
3. SYSTEM ANALYSIS	7
3.1. Overview of System Analysis	7
3.2. Dataset	10
3.3. System Requirements	11
4. SYSTEM DESIGN	12
4.1. Overview of System Design	12
4.2. Methodology	13
5. CODING & IMPLEMENTATION	25
6. SYSTEM TESTING	29
7. RESULTS	32
8. CONCLUSION	39
9. REFERENCES	40

LIST OF FIGURES

S.NO	DESCRIPTION	PAGENO
1	Google collab environment	8
2	System Architecture	12
3	Splitting the dataset into training, testing, validation datasets	14
4	The Model detection as a regression problem	17
5	YOLO model with Backbone architecture	17
6	Analysis of YOLOv8: A) The quantity of features in various YOLO architecture versions, B) The rate of inference for every YOLO version.	20
7	Performance Metrics of YOLOv8 on COCO dataset	20
8	Precision Vs Confidence curve for all types of animals	34
9	Recall vs Confidence curve for all types of animals	34
10	Precision vs Recall curve for all types of animals	35
11	Curve with Confidence for all types of animals	35
12	Labels Information chart for all animals	36
13	Labels Correlogram	36
14	Performance Evaluation	37
15	Confusion Matrix	37
16	Model prediction Results	38

LIST OF TABLES

S.NO	Tables	PAGENO
1	Performance Metrics of YOLOv8 on COCO dataset	20
2.	Comparision of model outcomes	38

1. INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

In the ever-evolving domain of wildlife management and conflict resolution, the imperative to devise intelligent systems has become more pronounced than ever. This project focuses on developing an Automated System for Detecting and Repelling Wild Animal Intrusions to mitigate human-wildlife conflicts. It leverages advanced technologies such as the YOLOv8 object detection model for real-time detection and Roboflow for efficient data preparation. By incorporating diverse and well-annotated datasets, the system aims to improve adaptability to various wildlife scenarios. Integration with wireless sensor networks enhances responsiveness and adaptability through real-time communication. Additionally, the project explores proactive repellent mechanisms inspired by recent advancements in intelligent animal repelling systems. This holistic approach aims to shift from detection-only systems to preventive measures, reducing harmful encounters between humans and animals. The project not only addresses the limitations of existing systems but also aims to redefine wildlife management by fostering harmonious coexistence between humans and animals.

1.2 FEASIBILITY STUDY

All of the projects that are available for execution, is given unlimited resources and infinite time. However, the development of the software is due to a lack of resources, and a complex supply rates. It has been a necessary and rational, to assess the feasibility of the project in the shortest amount of time possible.

1.2.1 Economic Feasibility

Economic feasibility attempts to weigh the costs of developing and implementing a new system, against the benefits that would accumulate from having the new system in place. A simple economic analysis which gives the actual comparison of costs and benefits are much more meaningful in the case.

1.2.2 Technical Feasibility

This study concentrates on the system's software and hardware and to what extent it can support the proposed system. The proposed system is technically feasible, since the equipment, software technology and available personal in the organization are well suitable for the proposed system.

1.2.3 Operational Feasibility

Conduct an operational feasibility study to assess the practicality of implementing animal deterrence with computer vision and Raspberry Pi. Evaluate factors such as hardware compatibility, power consumption, and real-world effectiveness. Determine the feasibility of deploying and maintaining the system in various environmental conditions.

1.3 SCOPE

The scope includes developing a system utilizing computer vision on Raspberry Pi to detect and deter animals. It involves designing algorithms for image recognition, integrating hardware components for real-time processing, and testing the effectiveness of deterrent mechanisms. The goal is to create a scalable solution for mitigating human-wildlife conflicts in diverse environments.

2. LITERATURE SURVEY

The escalating interaction between human habitats and natural ecosystems has intensified the urgency for innovative strategies to mitigate human-wildlife conflicts. A multitude of studies has explored the realm of animal intrusion detection and deterrence, leveraging technologies spanning from AI to the IOT. This literature review aims to synthesize key findings from pertinent studies, providing insights into contemporary strategies and aligning them with our project objectives.

Automated Wild-Animal Intrusion Detection and Repellent Systems: Patil and Ansari (2021) proposed an innovative system utilizing Artificial Intelligence of Things (AIoT) for wild-animal intrusion detection [1]. Their work underscores the integration of AI capabilities with conventional surveillance systems of wildlife detection. This aligns with our project's foundation, incorporating YOLOv8, an AI-driven object detection system, to dynamically identify and categorize potential threats [3].

Deep Learning for Animal Detection and Collision Avoidance: The field of deep learning for animal recognition and collision avoidance was investigated by Saxena et al. (2020) [2]. Their research emphasizes the significance of advanced computational techniques in preventing collisions between animals and vehicles. This resonates with our project's objectives, where YOLOv8, An advanced deep learning model, plays a pivotal role in real-time object detection and collision risk assessment [10].

Preventing Deer-Vehicle Accidents via Deep Learning Techniques: Fan, Sadeghian, and Aram (2020) focused on preventing deer-vehicle collisions through the application of deep learning techniques [3]. Their study highlights the potential of AI in predicting and preventing wildlife-related accidents. Our project builds upon this notion, implementing YOLOv8's robust object detection capabilities to identify animals in man-made environments and prevent potential collisions [10].

A Review on Deep Learning for Object Detection: A thorough analysis of deep learning-based object identification was presented by Zhao et al. (2019) [4]. Their insights into the evolution and applications of deep learning in object detection serve as a foundational reference for our project, which heavily relies on YOLOv8's object detection prowess [10].

Animal Detection using Deep Learning Algorithm: Banupriya et al. (2020) contributed to the domain with a focus on animal detection using deep learning algorithms [8]. Their research emphasizes the need for accurate and efficient algorithms in wildlife monitoring. Our project aligns with this perspective, employing YOLOv8 for its capability to rapidly and accurately detect animals [10].

Using RASPBERRYPI and Machine Learning for Wildlife Monitoring in Zoological Parks: Rasool and Murthy (2019) investigated how to integrate machine learning and Raspberry Pi for wildlife monitoring in zoological parks [6]. Their work highlights the feasibility of edge computing in wildlife surveillance. While our project doesn't directly involve Raspberry Pi, the underlying principle of edge-based processing aligns with the broader concept of intelligent, localized animal detection and deterrence.

A Review of Deeper Training-Based Object Identification Methods: the Buddha and Shyna (2019) offered a thorough summary of object-reconnaissance methods using neural networks [7]. Their insights into the landscape of dl methodologies for object detection contribute to the theoretical foundation of our project, which harnesses YOLOv8 for its efficiency and accuracy [10].

Singh et al. (2020) conducted a thorough investigation on animal identification: in man-made environments. They emphasised the significance of precise detection for a range of applications [10]. This resonates with our project's goal of implementing YOLOv8 for detecting animals in diverse scenarios, facilitating effective deterrence strategies.

Environmental Video Capture Dataset: Schneider's, Taylor, who is and Kremer (2018) investigated deep learning object identification techniques for environmental video traps information [11]. Their work contributes to the understanding of applying deep learning in ecological monitoring, which aligns with our project's utilization of AI for camera-based wildlife detection [10].

Utilising the Internet of Things to Track the well-being of animals: A Review: Nikam et al. (2018) conducted a review on animal welfare monitoring using the Internet of Things (IoT) [12]. Although our project doesn't explicitly focus on animal welfare, the reference to IoT in their work aligns with the broader trend of integrating technology for intelligent animal care, a principle relevant to our IoT-based approach [9].

In conclusion, this literature review synthesizes findings from diverse studies, providing a robust foundation for our project. The integration of YOLOv8, AIoT, and advanced data preparation tools represents a cohesive response to the challenges outlined in the reviewed literature. As we progress, these insights will inform the development of an intelligent animal deterrence system, contributing to the evolving landscape of human-wildlife conflict resolution.

2.1 EXISTINGSYSTEM

The existing system utilizes the COCO (Common Objects in Context) dataset for object detection tasks. COCO is a widely used dataset in the field of computer vision, containing a diverse range of object categories and annotations for object detection, segmentation, and captioning tasks. The dataset consists of 330,000 images, with annotations available for 200,000 images. It covers 80 object categories, including common objects like cars, bicycles, and animals, as well as more specific categories such as umbrellas, handbags, and sports equipment. Annotations provided include object bounding boxes, segmentation masks, and captions for each image. Additionally, COCO offers standardized evaluation metrics such as mean Average Precision (mAP) for object detection and mean Average Recall (mAR) for segmentation tasks, making it suitable for benchmarking model performance.

DRAWBACKS :

- 1. Dataset Limitations:** While COCO is a widely used and comprehensive dataset, it has limitations in terms of the number of object categories and instances. With only 80 categories and 330,000 object instances, the dataset may not cover all the diverse scenarios encountered in real-world applications. This limitation can impact the model's ability to generalize to unseen data and may result in suboptimal performance, particularly in niche or specialized domains where the object categories are not adequately represented.
- 2. Model Choice:** The existing system employs YOLOv3 with the Darknet architecture for object detection tasks. While YOLOv3 is an improvement over previous versions with features like multi-scale detection and a stronger feature extractor network, it still has limitations.

The choice of YOLOv3 may not be optimal for all scenarios, as its architecture and design decisions may not fully align with the specific requirements and constraints of the problem domain. Additionally, YOLOv3's performance may vary

depending on factors such as dataset characteristics, training data quality, and computational resources available for training and inference.

2.2 PROPOSED SYSTEM

The proposed system for animal deterrence using computer vision and Raspberry Pi integrates image recognition algorithms with hardware components to detect and deter animals. Utilizing a camera module connected to the Raspberry Pi, the system captures live video footage. This footage is processed in real-time using computer vision algorithms such as YOLO (You Only Look Once) or OpenCV (Open Source Computer Vision Library) to identify animals based on predefined criteria such as size, shape, and movement patterns.

Upon detecting an animal within the monitored area, the system triggers deterrent mechanisms. These mechanisms may include emitting sound or light signals, activating physical barriers like motion-activated sprinklers or ultrasonic devices, or sending alerts to human operators for intervention. Additionally, the system may utilize machine learning techniques to adaptively learn and improve its detection accuracy over time.

The Raspberry Pi serves as the central processing unit, providing computational power for image processing and decision-making. It also facilitates connectivity with external devices for triggering deterrent actions and transmitting alerts. The system can be deployed in various environments, such as agricultural fields, residential properties, or wildlife reserves, to mitigate human-wildlife conflicts effectively while minimizing false alarms and environmental impact.

3. SYSTEM ANALYSIS

3.1 OVERVIEW OF SYSTEM ANALYSIS

3.1.1 Computer Vision

Computer vision is a field of artificial intelligence that enables computers to interpret and understand visual information from the world around them. It involves developing algorithms and systems that can automatically extract, analyze, and comprehend useful information from images or videos. One of its fundamental goals is to replicate the human vision system's ability to perceive and interpret visual data. In practical terms, computer vision finds applications in various domains, including autonomous vehicles, healthcare, surveillance, augmented reality, and industrial automation. For instance, in autonomous vehicles, computer vision systems are used to identify and track objects such as pedestrians, vehicles, and traffic signs, enabling the vehicle to navigate safely. In healthcare, computer vision aids in medical imaging analysis, assisting doctors in diagnosing diseases from X-rays, MRI scans, and other medical images. These examples illustrate how computer vision revolutionizes industries by providing intelligent visual perception and analysis capabilities.

3.2 SOFTWARE USED IN THE PROJECT

3.2.1 Python

Python is an interpreted, high-level, general-purpose programming language. It is a multi-paradigm programming language. It is a widely used high-level programming language for general-purpose programming. Apart from being open source programming language, python is a great object-oriented, interpreted, and interactive programming language. Python combines remarkable power with very clear syntax. It has modules, classes, exceptions, very high level dynamic data types, and dynamic typing. There are interfaces to many system calls and libraries, as well as to various windowing systems. Python is also usable as an extension language for applications written in other languages that need easy-to-use scripting or automation interfaces. Python is widely considered as the preferred language for teaching and learning ML (Machine Learning). Few simple reasons are:

- It's simple to learn. As compared to C, C++ and Java the syntax is simpler and Python also consists of a lot of code libraries for ease of use.

- Though it is slower than some of the other languages, the data handling capacity is great.
- Open Source: – Python along with R is gaining momentum and popularity in the Analytics domain since both of these languages are opensource.
- Capability of interacting with almost all the third party languages and platforms

3.2.2 Google collab

Google Colab, short for Google Colaboratory, is a free cloud-based platform provided by Google that allows users to write and execute Python code in a Jupyter Notebook environment. It offers seamless integration with Google Drive, enabling users to store, share, and collaborate on notebooks effortlessly. Colab provides access to powerful hardware accelerators like GPUs and TPUs, making it ideal for training deep learning models and running computationally intensive tasks. Users can install and import libraries, visualize data, and generate interactive plots within the notebook interface. With its built-in support for Markdown, code snippets, and LaTeX equations, Google Colab facilitates the creation of interactive and informative documents. Additionally, Colab supports version control through integration with Git, allowing users to manage and track changes to their notebooks efficiently.

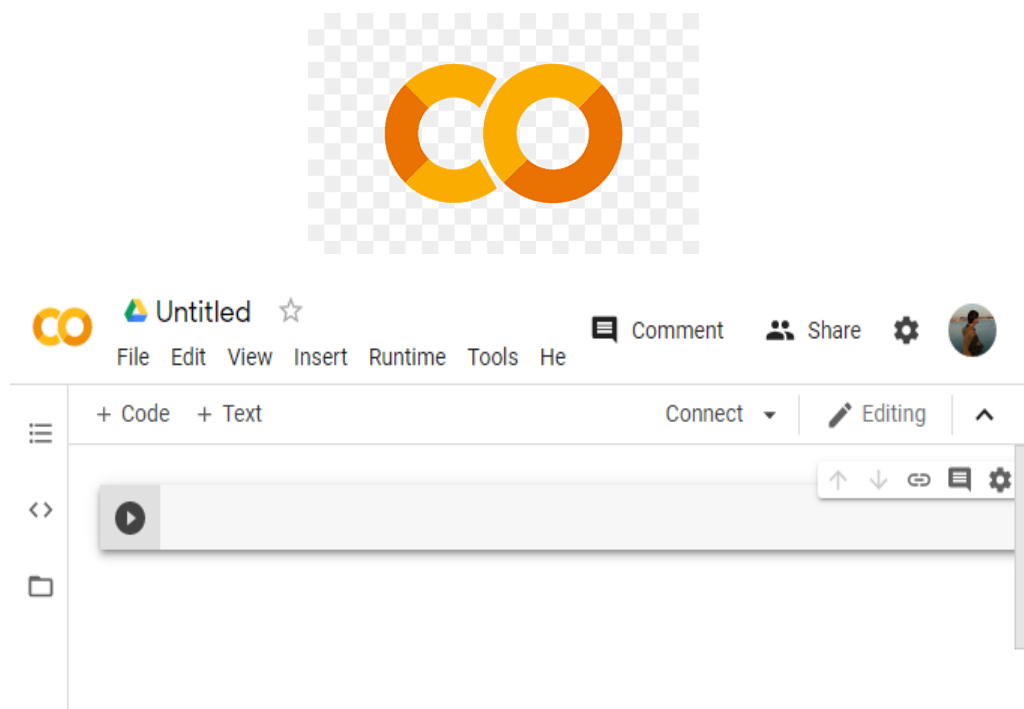


Figure 3.1: Google collab environment

3.2.3 Packages and Frameworks

Ultralytics Framework: Ultralytics, as a computer vision framework, builds upon the success and advancements of YOLOv5, providing enhanced capabilities and features. Its development reflects a commitment to improving object detection, image classification, and other computer vision tasks through innovative techniques and methodologies. With Ultralytics, users can expect optimizations for performance, efficiency, and accuracy, enabling faster inference times and better results. The framework likely includes tools for data preprocessing, model training, evaluation, and deployment, streamlining the entire computer vision pipeline. Moreover, Ultralytics may incorporate state-of-the-art models, architectures, and algorithms, ensuring that users have access to cutting-edge solutions for their computer vision projects. As a result, it holds promise for researchers, developers, and practitioners seeking to leverage the latest advancements in deep learning and computer vision technology.

Roboflow : Roboflow is an end-to-end computer vision platform that simplifies the process of building computer vision models. Various features of the roboflow are Dataset Management,

Model training, Hosted Inference, Edge Deployment , Access security and Rich support

pandas: pandas is a software library written for the Python programming language for data manipulation and analysis. In particular, it offers data structures and operations for manipulating numerical tables and time series. Pandas is mainly used for machine learning in form of dataframes. Pandas allow importing data of various file formats such as csv, excel etc

matplotlib.pyplot: It provides an object-oriented API for embedding plots into applications using general-purpose GUI toolkits. pyplot is a shell-like interface to Matplotlib. Pyplot maintains state across calls. It is useful for use in Jupyter or IPython notebooks.

sklearn: sklearn (Scikit-learn) is a free software machine learning library for the Python programming language. It features various classification, regression and clustering algorithms. It provides a range of supervised and unsupervised learning algorithms via a consistent interface in python.

3.2.4 Dataset

Dataset Description for YOLOv8 Model Training

A YOLOv8 models will be developed and trained as part of our research in order to provide an autonomous system for detecting and repelling wild animal intrusions. The quality and variety of the training dataset are critical to our algorithm's performance. Within this segment, we provide a comprehensive description of the dataset utilized, ensuring transparency and reproducibility in our research.

1. Dataset Origin and Collection:

- The dataset used for training our YOLOv8 model was meticulously collected from various sources, encompassing both wildlife reserves and natural habitats prone to human-animal conflicts. Special emphasis was given to include a diverse range of species, considering mammals, birds, and other wildlife commonly encountered in regions where our system is intended to be deployed.

2. Image Annotation:

- Each image in the dataset underwent a rigorous annotation process to create bounding boxes around the animals present. The annotation process involved the precise delineation of the animals to facilitate accurate training of the YOLOv8 model. This meticulous annotation ensures that the model can detect and classify animals with high precision in real-world scenarios.

3. Species Diversity:

- To enhance the model's adaptability, the dataset comprises of four animal species. This includes elephants, tigers, pigs and Leopards. The inclusion of diverse species ensures that the YOLOv8 model is capable of effectively detecting and responding to different types of wildlife.

4. Environmental Context:

- An essential aspect of our dataset is the incorporation of environmental context. Images were captured in various environmental conditions, including different times of the day, weather scenarios, and habitat types. This contextual diversity aids in training the model to make informed decisions, considering the broader ecological setting in which the automated system will operate.

5. Data Augmentation Techniques:

- During the training phase, data enhancement approaches were employed to increase a model's resilience and avoid excessive fitting. These methods include arbitrary flips, rotations, and lighting and contrast adjustments. By adding more illumination and orientation variables to the dataset, the model will be more likely to generalise effectively to unobserved real-world circumstances.

6. Model Training Parameters:

- Using the annotated dataset and specially designed parameters to maximise performance, the YOLOv8 model was trained. The training process involved 100 epochs, with an image size of 640x640 pixels. The choice of these parameters was based on iterative experimentation to strike a compromise between computing efficiency and precision.

7. Evaluation and Validation:

- A subset of the dataset was reserved for evaluation and validation purposes. The remainder was not utilised in the learning stage and acts as a separate benchmark to measure the model's performance. Measures like precision, recall, and F1 score were used to assess how well the algorithm identified and categorised creatures.

In Note, our dataset for YOLOv8 model training is a meticulously curated collection that reflects the diverse scenarios our system is likely to encounter. The inclusion of various species, environmental contexts, and the application of data augmentation techniques ensures that our model is robust, adaptable, and capable of delivering accurate results in real-world situations.

3.3 SYSTEM REQUIREMENTS

3.3.1 Software Requirements

Operating System	:	Raspberry Pi 4 OS (64 bit)
Languages	:	Python
IDE	:	Google collab

3.3.2 Hardware Requirements

RAM	:	Min 4 GB
Processor	:	Intel i5 or Above
Hard disk	:	512 SSD
Raspberry pi Memory	:	32 GB
Motor Controller	:	L298 Bridge
Ultrasonic Sensor	:	HC- SR04
Dc Motor	:	3 volts
Battery	:	12 Volts
Raspberry pi camera module	:	-

4. SYSTEM DESIGN

4.1 OVERVIEW OF SYSTEMDESIGN

The Wild-Animal Intrusion Detection and Repellent System integrates multiple modules to tackle the challenges arising from human-wildlife conflicts. Beginning with the Data Collection Module, a network of strategically positioned cameras and sensors captures images and videos of the monitored area. These data undergo preprocessing and augmentation to enhance quality and ensure the robustness of the dataset.

Central to the system is the YOLOv8 Model, which handles object detection using transfer learning techniques for improved performance, especially in scenarios with limited labeled data. Model Training and Optimization further refine the YOLOv8 model, optimizing parameters and conducting hyperparameter tuning to adapt effectively to the monitored environment.

Once trained, the model is deployed for real-time Detection and Classification of wild animals, with additional modules identifying species using deep learning techniques. The Decision-Making Module filters out false positives through threshold analysis and generates alerts upon detecting potential intrusions.

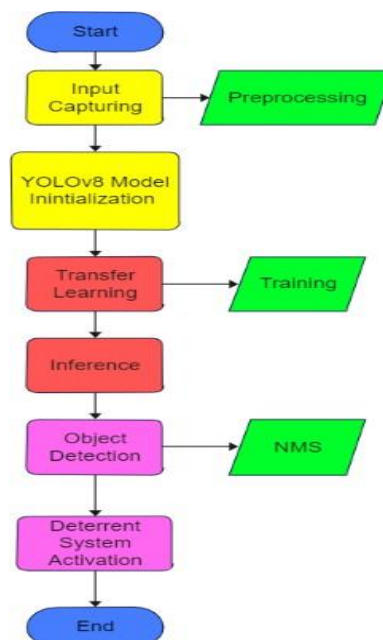


Figure 4.1: System Architecture.

Upon confirmation of an intrusion, the Repellent System Activation module comes into play, deterring animals from the monitored area with variable intensity based on species and behavior. Finally, the Monitoring and Logging system records detection events, system responses, and anomalies, providing users with real-time updates and insights through a dedicated interface.

This holistic architecture prioritizes real-time detection, effective deterrence, and user-friendly control interfaces, aiming to mitigate human-wildlife conflicts and foster harmonious coexistence between humans and animals.

4.2 METHODOLOGY

The proposed system uses YOLOv8 model in order to detect the animals in real time scenarios and deter the animals by making sounds without harming them. It's been integrated with a Micro Processor Raspberry Pi to detect the animal in the frame and it will give some sound to deter the animal.

The proposed algorithm breaks down the procedure step-by-step as follows.

Step 1: Gather Data for the Model. We have gathered various animal images like elephant, tiger, leopard and pig in different environmental context. So that the model can learn and detect the animals in all kind of environments.

Step 2: Upload and Annotate Images of animals in Roboflow which is a platform for various object detection tasks like Data collection, Data Annotation, Model training and Model Deployment

Step 3: Generate a Dataset Version. Once the data annotation was done the we had created a dataset version on which the model will be trained. We had used bounding box type of annotation for this project

Step 4: After creation of proper dataset version we have to train the yolov8 model on our animal dataset by using google collab with the support of GPUs and once the model completed its training it will generate weights which we need to store for detecting.

Step 5: Finally, We have to deploy the model on to Raspberry pi along with its camera module

4.2.1 Splitting the dataset

Training, Validation, and Test Sets

Splitting your dataset is essential for an unbiased evaluation of prediction performance. In most cases, it's enough to split your dataset randomly into three subsets

1. **The training set** is applied to train, or **fit**, your model. For example, you use the training set to find the optimal weights, or coefficients, for Linear Regression, Logistic regression, or Neural Networks.
2. **The validation set** is used for unbiased model evaluation during Hyperparameter tuning. For example, when you want to find the optimal number of neurons in a neural network or the best kernel for a support vector machine, you experiment with different values. For each considered setting of hyperparameters, you fit the model with the training set and assess its performance with the validation set.
3. **The test set** is needed for an unbiased evaluation of the final model. You shouldn't use it for fitting or validation.

In less complex cases, when you don't have to tune hyperparameters, it's okay to work with only the training and test sets.

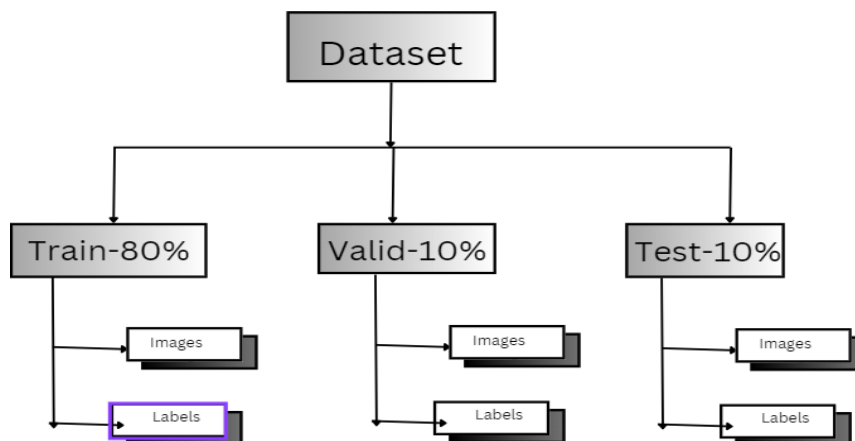


Figure 4.2.1: Splitting the dataset into training, testing, validation datasets

4.2.2 YOLO OBJECT DETECTION ALGORITHMS :

Joseph Redmon et al. introduced the YOLO (You Only Look Once) algorithm for real-time object detection in 2016. The YOLO algorithm utilizes a single neural network for predicting the class probabilities and bounding boxes of objects in an image. Traditional object recognition algorithms have multiple steps to find objects, but YOLO only has one step to process the whole picture. This makes it faster and more efficient. The YOLO algorithm also uses anchor boxes to improve detection accuracy and handle objects of different sizes and aspect ratios. YOLOv8, which forms the latest version of the algorithm, is the fastest and most accurate YOLO model to date, which achieves state-of-the-art evaluations on several benchmark datasets.

Ultralytics YOLOv8 is not just another object detection model; it's a versatile framework designed to cover the entire lifecycle of machine learning models—from data ingestion and model training to validation, deployment, and real-world tracking. Each mode serves a specific purpose and is engineered to offer you the flexibility and efficiency required for different tasks and use-cases.

YOLO: A Brief History

- YOLO (You Only Look Once), a popular object detection and image segmentation model, was developed by Joseph Redmon and Ali Farhadi at the University of Washington. Launched in 2015, YOLO quickly gained popularity for its high speed and accuracy
- YOLOv2, released in 2016, improved the original model by incorporating batch normalization, anchor boxes, and dimension clusters
- YOLOv3, launched in 2018, further enhanced the model's performance using a more efficient backbone network, multiple anchors and spatial pyramid pooling..
- YOLOv4 was released in 2020, introducing innovations like Mosaic data augmentation, a new anchor-free detection head, and a new loss function.
- YOLOv5 further improved the model's performance and added new features such as hyperparameter optimization, integrated experiment tracking and automatic export to popular export formats.
- YOLOv6 was open-sourced by Meituan in 2022 and is in use in many of the company's autonomous delivery robots.
- YOLOv7 added additional tasks such as pose estimation on the COCO keypoints dataset.
- YOLOv8 is the latest version of YOLO by Ultralytics. As a cutting-edge, state-of-the-art (SOTA) model, YOLOv8 builds on the success of previous versions, introducing new features and improvements for enhanced performance, flexibility, and efficiency. YOLOv8 supports a full range of vision AI tasks, including detection, segmentation, pose estimation.

YOLO unify the separate components of object detection into a single neural network. Our network uses features from the entire image to predict each bounding box. It also predicts all bounding boxes across all classes for an image simultaneously. This means our network reasons globally about the full image and all the objects in the image. The YOLO design enables end-to-end training and real time speeds while maintaining high average precision. YOLO system divides the input image into an $S \times S$ grid. If the center of an object falls into a grid cell, that grid cell is responsible for detecting that object. Each grid cell predicts B bounding boxes and confidence scores for those boxes. These confidence scores reflect how confident the model is that the box contains an object and also how accurate it thinks the box is that it predicts.

Formally we define confidence as

$$Pr(Object) * IOU_{truth\ pred}.$$

If no object exists in that cell, the confidence scores should be zero. Otherwise we want the confidence score to equal the intersection over union (IOU) between the predicted box and the ground truth. Each bounding box consists of 5 predictions: x , y , w , h , and confidence. The (x, y) coordinates represent the center of the box relative to the bounds of the grid cell. The width and height are predicted relative to the whole image.

Finally the confidence prediction represents the IOU between the predicted box and any ground truth box. Each grid cell also predicts C conditional class probabilities, $Pr(Class_i | Object)$.

These probabilities are conditioned on the grid cell containing an object. We only predict one set of class probabilities per grid cell, regardless of the number of boxes B .

At test time we multiply the conditional class probabilities and the individual box confidence predictions,

$$Pr(Class_i | Object) * Pr(Object) * IOU_{truth\ pred} = Pr(Class_i) * IOU_{truth\ pred} \quad (1)$$

which gives us class-specific confidence scores for each box. These scores encode both the probability of that class appearing in the box and how well the predicted box fits the object.

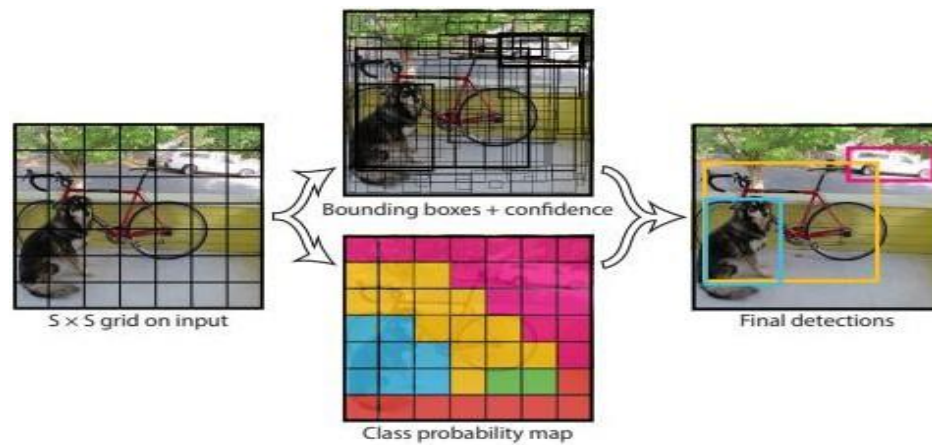


Figure 4.2.2: The Model detection as a regression problem. It divides the image into an $S \times S$ grid and for each grid cell predicts B bounding boxes, confidence for those boxes, and C class probabilities. These predictions are encoded as an $S \times S \times (B * 5 + C)$ tensor

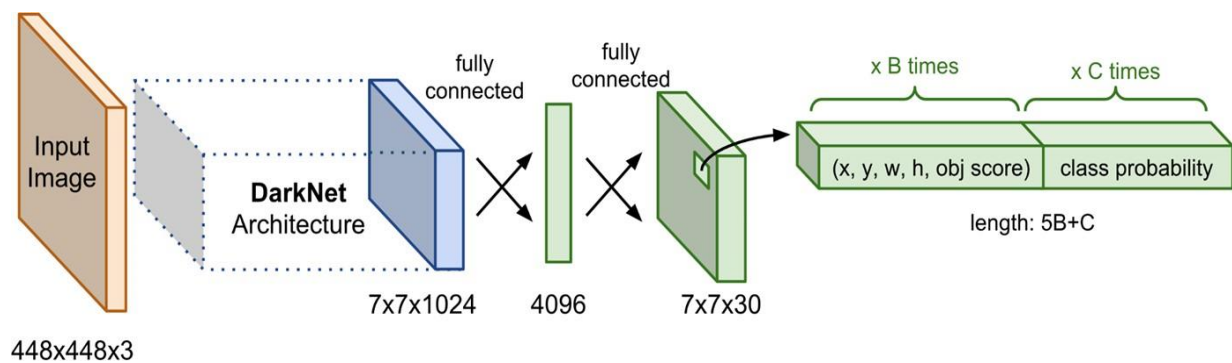


Figure 4.2.3: YOLO model with Backbone architecture

4.2.3 YOLOv8 Architecture:

The architecture of YOLOv8, or You Only Look Once version 8, is a state-of-the-art object detection model that builds upon the success of its predecessors. YOLOv8 is known for its efficiency and accuracy in real-time object detection tasks, making it widely adopted in various computer vision applications.

The architecture is designed to address the limitations of previous YOLO versions while maintaining a balance between speed and precision.

One notable improvement in YOLOv8 is its modular and scalable design. The model is divided into three main components: the backbone, neck, and head. The backbone is responsible for extracting features from the input image, and YOLOv8 employs a variety of backbones, including CSPDarknet53 and EfficientNet.

The neck connects the backbone to the head and is crucial for feature fusion. The head is responsible for predicting bounding boxes, object classes, and confidence scores.

Another key aspect of YOLOv8's architecture is its focus on model scaling. YOLOv8 offers different variants, such as YOLOv8-tiny and YOLOv8x, which vary in size and computational complexity. This allows users to choose a model that fits their specific requirements, whether it be for resource-constrained environments or high-performance applications.

YOLOv8 also introduces advancements in training strategies, incorporating techniques like Rectified Adam (RAdam) optimization and the use of anchor-based or anchor-free object detection. These improvements contribute to faster convergence during training and enhanced performance in object detection tasks.

Moreover, YOLOv8 has a flexible configuration system that allows users to easily customize various parameters, such as input size, anchor boxes, and model complexity. This flexibility makes YOLOv8 adaptable to diverse datasets and application scenarios.

The architecture of YOLOv8 is characterized by its modular design, scalable variants, improved backbones, and advanced training strategies. These features collectively contribute to its success in real-time object detection, making it a popular choice for researchers and practitioners in the field of computer vision.

YOLOv8 is the latest iteration in the YOLO series of real-time object detectors, offering cutting-edge performance in terms of accuracy and speed. Building upon the advancements of previous YOLO versions, YOLOv8 introduces new features and optimizations that make it an ideal choice for various object detection tasks in a wide range of applications.

4.2.4 Key Features of YOLOv8 Architecture:

YOLOv8, or You Only Look Once version 8, is an object detection model that builds upon its predecessors to improve accuracy and efficiency. Here are some key features of the YOLOv8 architecture:

1. Backbone Network:

YOLOv8 employs a feature-rich backbone network as its foundation. The network serves to extract hierarchical features from the input image, providing a comprehensive representation of the visual information.

YOLOv8 utilizes CSPDarknet53, a modified version of the Darknet architecture, as its backbone. This modification incorporates Cross Stage Partial networks, enhancing the learning capacity and efficiency.

2. Neck Architecture:

The architecture includes a novel neck structure, which is responsible for feature fusion. This is crucial for combining multi-scale information and improving the model's ability to detect objects of varying sizes.

YOLOv8 introduces PANet (Path Aggregation Network), a feature pyramid network that facilitates information flow across different scales. PANet enhances the model's ability to handle objects with diverse scales in a more effective manner.

3. YOLO Head:

YOLOv8 retains the characteristic feature of the YOLO series – the YOLO head. This component generates predictions based on the features extracted by the backbone network and the neck architecture.

The YOLO head predicts bounding box coordinates, objectness scores, and class probabilities for each anchor box associated with a grid cell. The architecture uses anchor boxes to efficiently predict objects of different shapes and sizes.

4. Training Techniques:

YOLOv8 leverages advancements in training strategies to improve convergence speed and model performance. MixUp, a data augmentation technique, is employed to create linear interpolations of images, enhancing the model's generalization capabilities.

Additionally, YOLOv8 utilizes a cosine annealing scheduler for learning rate adjustments during training, contributing to more stable convergence.

5. Model Variants:

YOLOv8 is available in different variants, each designed for specific use cases. YOLOv8-CSP, for instance, focuses on striking a balance between accuracy and speed. YOLOv8x-Mish, another variant, employs the Mish activation function for improved non-linearity, leading to better generalization and performance.

6. YOLOv8 Performance:

YOLOv8 has demonstrated remarkable improvements in terms of accuracy and speed compared to its predecessors. With superior real-time object detection capabilities, YOLOv8 has become a popular choice in various applications, including robotics, surveillance, and augmented reality.

These features collectively contribute to making YOLOv8 a versatile and efficient object detection model for various applications, from real-time systems to high-accuracy scenarios.

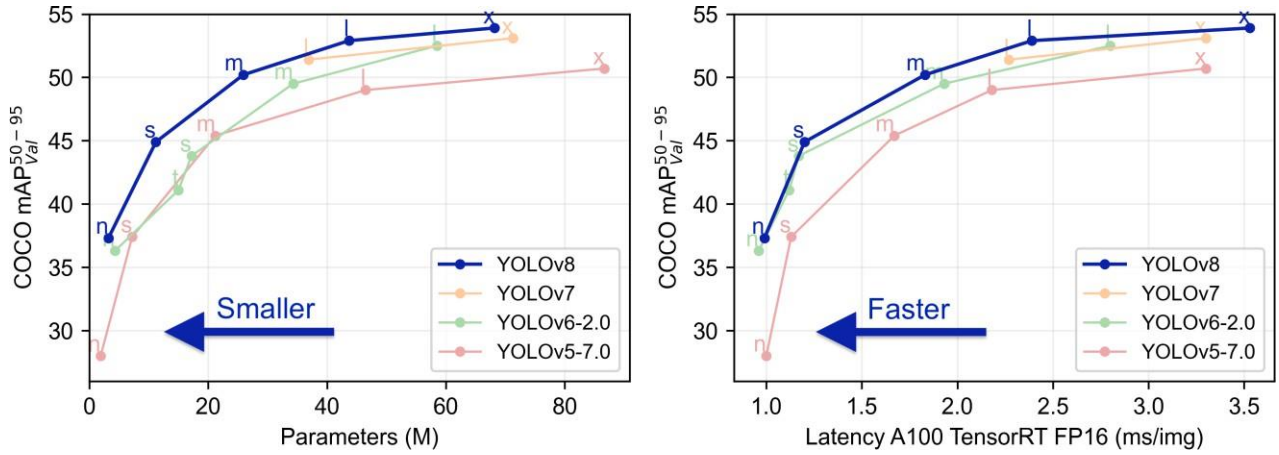


Figure:4.2.4 - Analysis of YOLOv8: A) The quantity of features in various YOLO architecture versions, B) The rate of inference for every Yolo version.

Model	Input size (pixels)	mAPbox 50-95	mAPmask 50-95	params (M)	FLOPS (B)
YOLOv8n	640	36.7	30.5	3.4	12.6
YOLOv8s	640	44.6	36.8	11.8	42.6
YOLOv8m	640	49.9	40.8	27.3	110.2
YOLOv8l	640	52.3	42.6	46.0	220.5
YOLOv8x	640	53.4	43.4	71.8	344.1

Table 4.2.5: Performance Metrics of YOLOv8 on COCO dataset

4.3 UML DIAGRAMS

In the field of software engineering, the Unified Modeling Language (UML) is a standardized visual specification language for object modeling. UML is a general-purpose modeling language that includes a graphical notation used to create an abstract model of a system, referred to as a UML model. The model also contains a “Semantic backplane”-documentation such as written use cases that drive the model elements and diagrams.

Importance of UML in Modeling:

A modeling language is a language whose vocabulary and rules focus on the conceptual and physical representation of a system. A modeling language such as UML is thus a standard language for software blueprints. The UML is not a visual programming language, but its models can be directly connected to various programming languages. This means that it is possible to map from a model in the UML to a programming language Java, C++ or Visual Basic, or even to tables in relational database or the persistent store of an object oriented database. This mapping permits forward engineering: the generation of code from a UML model into a programming language. The reverse is also possible you can reconstruct a model from an implementation back into UML. This is a programming language that is used for object-oriented software development. To organize program code more efficiently, programmers often create “objects” that are sets of structured data within programs. UML, which has been standardized by the Object Management Group (OMG), was designed for this purpose. The language has gained enough support that it has become a standard language for visualizing and constructing software programs.

A conceptual model of UML:

The three major elements of UML are

- The UML’s basic building blocks
- The rules that dictate how those building blocks may be put together
- Some common mechanism that apply throughout the UML

4.3.1 Use case Diagram:

A use case diagram shows a set of use cases and actors and their relationships. We apply

the use case diagram to illustrate the static use case view of a system. Use case diagram are especially important in organizing and modeling the behaviors of a system.

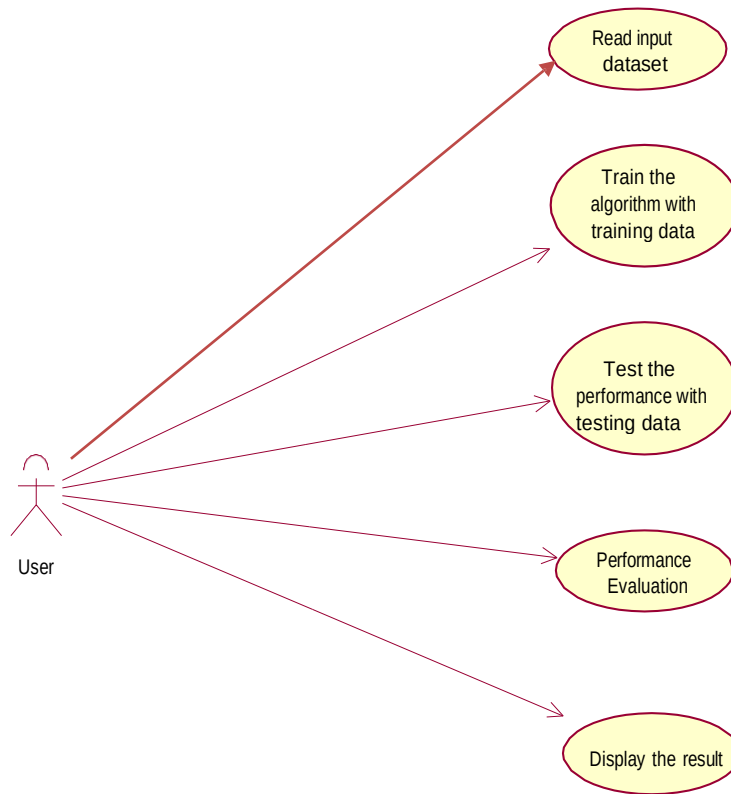


Figure 4.3.1: Use Case Diagram

4.3.2 Class Diagram:

Classes are the most important building block of any object oriented system. A class is a description of set of objects that share the same attributes, operations, relationships and semantics. A class implements one or more interfaces. It is graphically rendered as a rectangle.

IDENTIFYING RELATIONSHIPS:

In UML the ways that things can connect to each other wither logically or physically are modeled as relationships. In object-oriented modeling there are three kinds of relationships that are most important, they are

1. **Dependencies:** It is a using relationship that states that a change in specification of one

thing may affect another thing that uses each. Graphically dependency is rendered as a dashed directed line.

2. **Generalization:** It is relationship between a general thing (called parent) and a more specific kind of that thing (called the child). Generalization is sometimes called ‘is-a-kind-of’ relationship. It means that objects of the child may be used anywhere that parent may appear. Graphically it is rendered as a solid directed line with a large open arrow head pointing to the parent.
3. **Association:** It is a structural relationship that specifies that objects of one thing are connected to the objects of another. The associations that connect more than two classes are called n-array associations. Graphically it is rendered as solid line connecting the same or different classes.

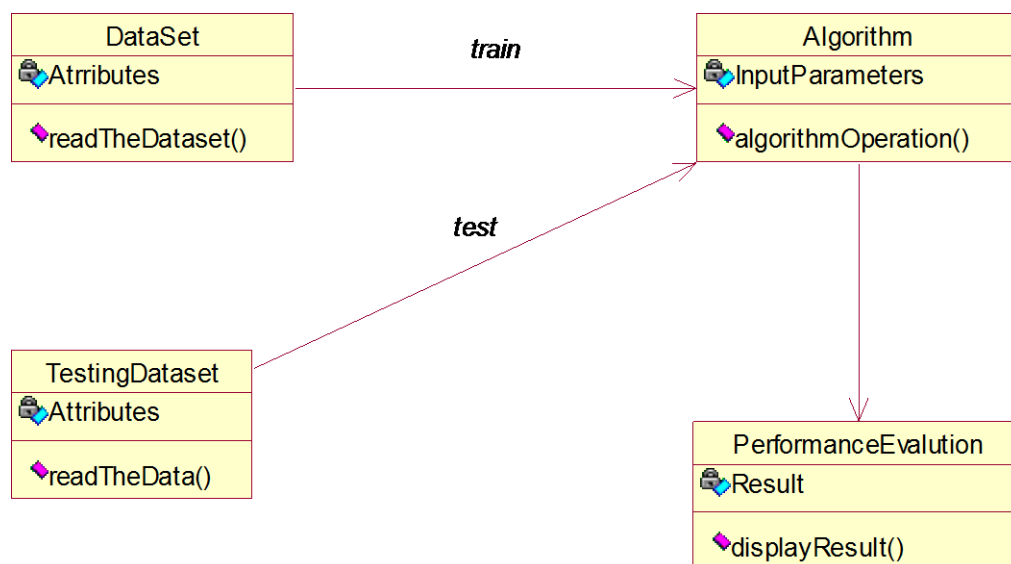


Figure 4.3.2: Class Diagram

4.3.3 Activity Diagram:

The purpose of an activity diagram is to provide a view of flows and what is going on inside a use case or among several classes. An activity diagram is just to explain the internal

operations performed and also the transitions that are triggered by the completion of the particular operations. At the abstract level it explains the sequence of the activities. This focus on the events occurring to a single object as it responds to messages, an activity diagram can be used to model an entire process. Activity diagram commonly contains:

Activity states and action states

Calling an operation on an object sending a signal to an object or even creating or destroying an object, these executable computations are called action states because they are the states of the system, representing the execution of an action.

Activity states can be further decomposed, their activity being represented by other activity diagrams these may be interrupted and take some duration to complete.

Transitions

When the action or activity of a state completes flow of control passes immediately to the next action or activity state. You specify this flow by using transitions to show the path from one action or activity state to next action or activity state. You represent this as simple directed line.

Object Flow

Objects may be involved in the flow of control associated with an activity diagram. The use of dependency relationships and objects is called an object flow because it represents participation of an object in the flow of control

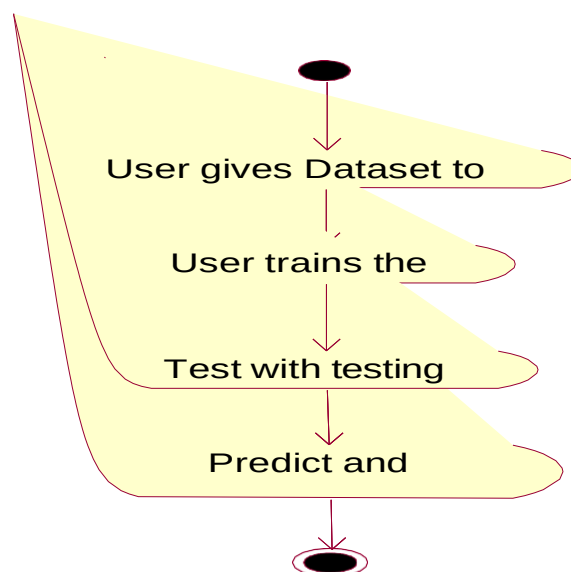


Figure 4.3.3: Activity Diagram

5. CODING AND IMPLEMENTATION

Various Modes of using YOLOv8 model

Understanding the different modes that Ultralytics YOLOv8 supports is critical to getting the most out of your models:

- Train mode: Fine-tune your model on custom or preloaded datasets.
- Val mode: A post-training checkpoint to validate model performance.
- Predict mode: Unleash the predictive power of your model on real-world data.
- Export mode: Make your model deployment-ready in various formats.
- Track mode: Extend your object detection model into real-time tracking applications.
- Benchmark mode: Analyze the speed and accuracy of your model in diverse deployment environments.

Step 1:Installing are required dependences by using requirements.txt file

Required dependencies

```
dependencies = [  
    "matplotlib>=3.3.0",  
    "opencv-python>=4.6.0",  
    "pillow>=7.1.2",  
    "pyyaml>=5.3.1",  
    "requests>=2.23.0",  
    "scipy>=1.4.1",  
    "torch>=1.8.0",  
    "torchvision>=0.9.0",  
    "tqdm>=4.64.0", # progress bars  
    "psutil", # system utilization  
    "py-cpuinfo", # display CPU info  
    "thop>=0.1.1", # FLOPs computation  
    "pandas>=1.1.4",  
    "seaborn>=0.11.0", # plotting  
]
```

Step 2: Model Training

Training YOLOv8 model on custom dataset which contains animal species like Elephant, Tiger, Leopard, Pig. The training can be done in different environments we had used single GPU or CPU based training in Google collab. Train YOLOv8n on the animal dataset for 50 epochs at image size 640. The training device can be specified using the `device` argument. If no argument is passed GPU `device=0` will be used if available, otherwise `device=cpu` will be used.

Single-GPU and CPU Training in python

```
from ultralytics import YOLO
```

```
# Load a model
```

```
model = YOLO('yolov8s.yaml') # build a new model from YAML
```

```
model = YOLO('yolov8s.pt') # load a pretrained model (recommended for training)
```

```
model = YOLO('yolov8s.yaml').load('yolov8n.pt') # build from YAML and transfer weights
```

```
# Train the model
```

```
results = model.train(data='dataset.yaml', epochs=50, imgsz=640)
```

Step 3: Model Validation

Validation is a critical step in the machine learning pipeline, allowing you to assess the quality of your trained models. Val mode in Ultralytics YOLOv8 provides a robust suite of tools and metrics for evaluating the performance of your object detection models. This guide serves as a complete resource for understanding how to effectively use the Val mode to ensure that your models are both accurate and reliable.

```
from ultralytics import YOLO
```

```
# Load a model
```

```
model = YOLO('yolov8s.pt') # load an official model
```

```
model = YOLO('path/to/best.pt') # load a custom model
```

```
# Validate the model
```

```
metrics = model.val() # no arguments needed, dataset and settings remembered
```

```
metrics.box.map # map50-95
```

```
metrics.box.map50 # map50
```

```
metrics.box.map75 # map75
```

```
metrics.box.maps # a list contains map50-95 of each category
```

Step 4: Model Prediction

In the world of machine learning and computer vision, the process of making sense out of visual data is called 'inference' or 'prediction'. Ultralytics YOLOv8 offers a powerful feature known as **predict mode** that is tailored for high-performance, real-time inference on a wide range of data sources.

```
from ultralytics import YOLO

# Load a model
model = YOLO('yolov8n.pt') # pretrained YOLOv8n model

# Run batched inference on a list of images
results = model(['im1.jpg', 'im2.jpg']) # return a list of Results objects

# Process results list
for result in results:

    boxes = result.boxes # Boxes object for bounding box outputs

    masks = result.masks # Masks object for segmentation masks outputs

    keypoints = result.keypoints # Keypoints object for pose outputs

    probs = result.probs # Probs object for classification outputs

    result.show() # display to screen

    result.save(filename='result.jpg') # save to disk
```

Step 5: Running on Video for inference

```
import cv2

from ultralytics import YOLO

from google.colab.patches import cv2_imshow

# Load the YOLOv8 model
model = YOLO('runs/detect/train3/weights/best.pt')
```

```
# Open the video file

video_path = "/content/pig_-_106560 (540p).mp4"

cap = cv2.VideoCapture(video_path)

# Loop through the video frames

while cap.isOpened():

    # Read a frame from the video

    success, frame = cap.read()

    if success:

        # Run YOLOv8 inference on the frame

        results = model(frame)

        # Visualize the results on the frame

        annotated_frame = results[0].plot()

        # Display the annotated frame

        cv2.imshow(annotated_frame)

        # Break the loop if 'q' is pressed

        if cv2.waitKey(1) & 0xFF == ord("q"):

            break

        else:

            # Break the loop if the end of the video is reached

            break

    # Release the video capture object and close the display window

    cap.release()

cv2.destroyAllWindows()
```

6. SYSTEM TESTING

6.1 OVERVIEW OF TESTING

Software testing is the process used to assess the quality of computer software. Software testing is an empirical technical investigation conducted to provide stakeholders with information about the quality of the product or service under test, with respect to the context in which it is intended to operate. This includes, but is not limited to, the process of executing a program or application with the intent of finding software bugs. Quality is not an absolute; it is value to some person. With that in mind, testing can never completely establish the correctness of arbitrary computer software; testing furnishes a criticism or comparison that compares the state and behavior of the product against a specification. An important point is that software testing should be distinguished from the separate discipline of Software Quality Assurance (S.Q.A.), which encompasses all business process areas, not just testing.

6.2 TYPES OF TESTING METHODS

Software testing methods are traditionally divided into BLACK BOX TESTING and WHITE BOX TESTING. These two approaches are used to describe the point of view that a test engineer takes when designing test cases.

6.2.1 Black Box Testing

Black box testing treats the software as a black-box without any understanding of internal behavior. It aims to test the functionality according to the requirements. Thus, the tester inputs data and only sees the output from the test object. This level of testing usually requires thorough test cases to be provided to the tester who then can simply verify that for a given input, the output value (or behavior), is the same as the expected value specified in the test case. Black box testing methods include: equivalence partitioning, boundary value analysis, all-pairs testing, fuzz testing, model-based testing, traceability matrix etc.

6.2.2 White Box Testing

White box testing, however, is when the tester has access to the internal data structures, code, and algorithms. White box testing methods include creating tests to satisfy some code criteria. For example, the test designer can create tests to cause all statements in the program to be executed at least once. Other examples of white box testing are mutation testing and fault injection methods. White box testing includes all static testing.

Testing can be done on the following levels

6.2.3 Unit Testing:

Unit testing tests the minimal software component, or module. Each unit (basic component) of the software is tested to verify that the detailed design for the unit has been correctly implemented. In an object-oriented environment, this is usually at the class level, and the minimal unit tests include the constructors and destructors.

6.2.4 Integration Testing

Integration testing exposes defects in the interfaces and interaction between integrated components (modules). Progressively larger groups of tested software components corresponding to elements of the architectural design are integrated and tested until the software works as a system.

6.2.5 System Testing

System testing tests a completely integrated system to verify that it meets its requirements. System integration testing verifies that a system is integrated to any external or third party systems defined in the system requirements.

Test cases:

A test case is a software testing document, which consists of event, action, input, output, expected result and actual result. Clinically defined (IEEE 829-1998) a test case is an input and an expected result. This can be as pragmatic as 'for condition x your derived result is y', whereas other test cases described in more detail the input scenario and what results might be expected. It can occasionally be a series of steps (but often steps are contained in a separate test procedure that can be exercised against multiple test cases, as a matter of economy) but with one expected result or expected outcome.

A test case should also contain a place for the actual result. These steps can be stored in a word processor document, spreadsheet, database or other common repository. In a database system, you may also be able to see past test results and who generated the results and the system configuration used to generate those results. These past results would usually be stored in a separate table.

The term test script is the combination of a test case, test procedure and test data. Initially the term was derived from the byproduct of work created by automated regression test tools. Today, test scripts can be manual, automated or a combination of both.

The most common term for a collection of test cases is a test suite. The test suite often also contains more detailed instructions or goals for each collection of test cases. It definitely contains a section where the tester identifies the system configuration used during testing. A group of test cases may also contain prerequisite states or steps, and descriptions of the following tests.

7.RESULTS

The assessment of our object detection model encompassed a diverse dataset with various wildlife species, including Elephants, Leopards, Pigs, and Tigers. Essential performance metrics were employed to gauge the precision, recall, and overall efficacy of the model.

7.1. Performance Metrics:

- a. Precision (P):** The model demonstrated an overall precision of 89.8%, signifying high accuracy in positive predictions.
- b. Recall (R):** The overall recall stood at 76.1%, showcasing the model's effectiveness in capturing relevant instances.
- c. mean Average Precision (mAP) at 50% IoU (mAP50):** At a 50% IoU threshold, the model achieved an mAP of 86.2%, highlighting its robust performance in localization and identification.
- d. mAP between 50% and 95% IoU (mAP50-95):** The model showcased a mAP50-95 of 58.1%, indicating consistent object detection performance.

7.2. Class-Wise Results (Below will only act as a sample)

- i. Elephant: Precision: 88.5%, Recall: 90.6%, mAP: 95.1%.
- ii. Leopard: Precision: 100%, Recall: 55.6%, mAP: 71.9%
- iii. Pig: Precision: 95%, Recall: 68.2%, mAP: 83.8%
- iv. Tiger: Precision: 75.9%, Recall: 90.1%, mAP: 94.2%

7.3 Efficiency Metrics:

Speed: Preprocessing Time: 0.8ms, Inference Time: 4.7ms, Post-processing Time: 3.1ms.
The model demonstrated efficient processing times, making it suitable for real-time applications.

7.4 Comparative Analysis:

We contrast how well the suggested technique with current algorithms in the literature to assess its efficacy. Though there are several methods, the suggested CNN model demonstrates competitive accuracy and resilience in a variety of tasks, such as emotion identification, yawning detection, and sleep detection.

Formulae for Comparison:

1. **Accuracy:** Accuracy is one of the key metrics that can be ascertained from the confusion matrix. It demonstrates how well the classifier is overall. $Accuracy = (TP + TN) / (FP + FNTN + TN)$.
2. **The Precision:** An additional significant metric that can be obtained from the confusion matrix is precision. It gauges a classifier's capacity to correctly identify instances of one class without labeling them as belonging to another. $Precision = (TP) / (FP + TP)$
3. **Recall (Sensitivity):** A key performance indicator for classification models is recall, particularly when dealing with unbalanced datasets. $Recall = (TP) / (FN + TP)$
4. **AUC (Area Under the Curve):** An overall performance metric across all potential classification criteria is offered by AUC. $AUC = \int_0^1 \text{ROC curve}$.

7.5 Training Results :Graphs and Tables:

To visually represent the performance, we provide the following graphs:

1. **Bar Chart:** A bar chart comparing the accuracy, recall, and AUC of each model.
2. **Confusion Matrix:** For machine learning models, confusion matrices provide a comprehensive analysis of forecasts that are false, negative, genuine positive, and false negative.

7.5.1. Precesion Curve:

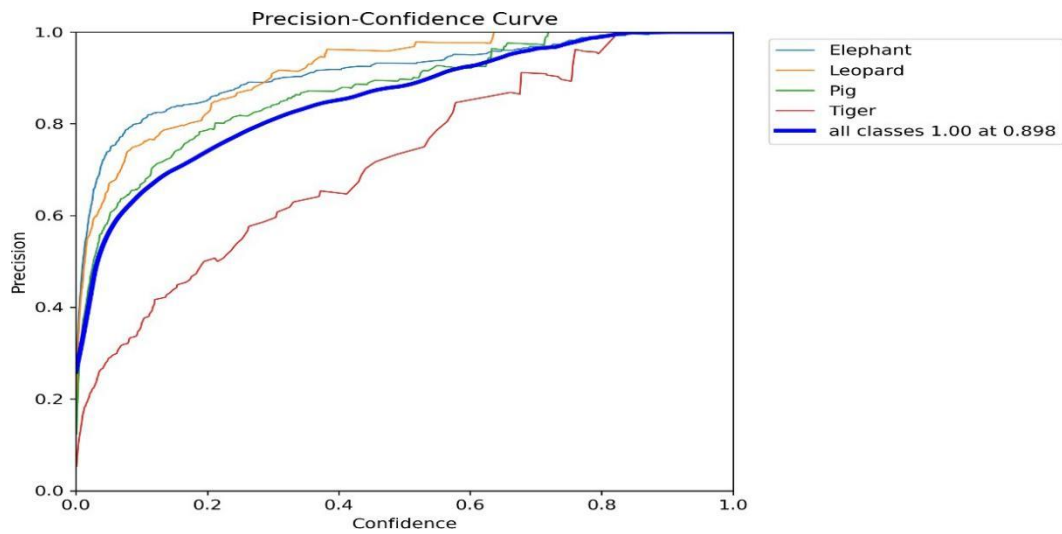


Figure:7.5.1 – Precision Vs Confidence curve for all types of animals

7.5.2. Recall Curve:

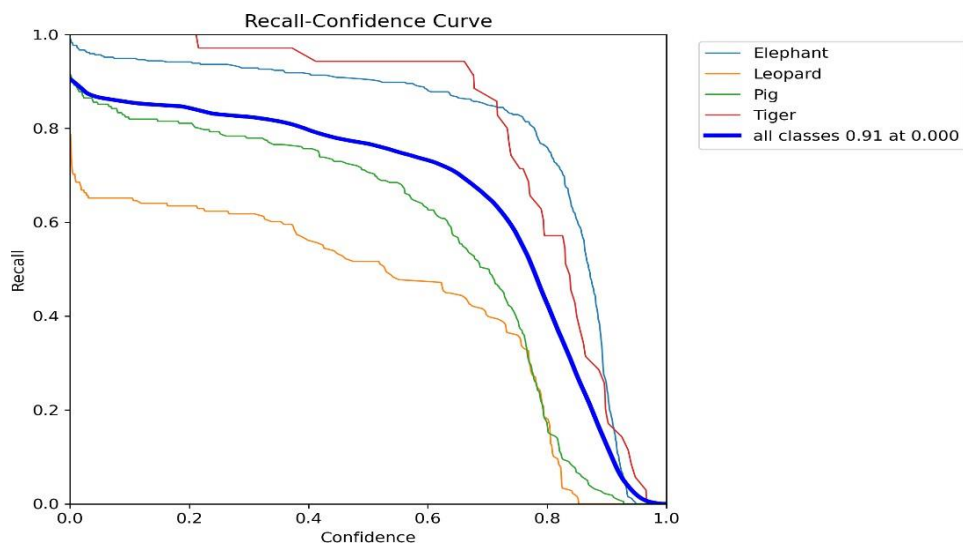


Figure:7.5.2 – Recall vs Confidence curve for all types of animals

7.5.3. Precision vs Recall Curve:

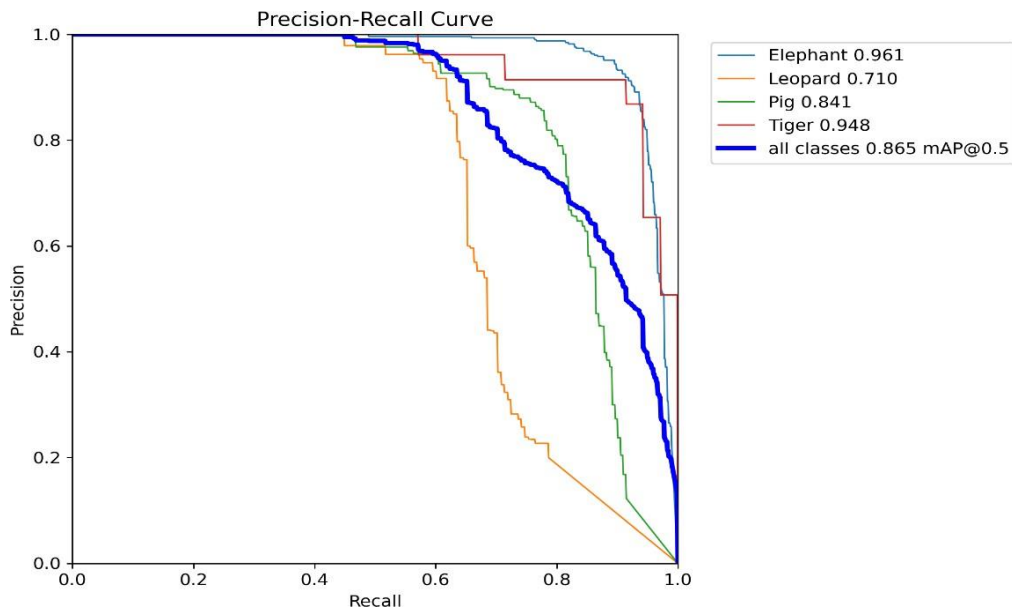


Figure:7.5.3 – Precision vs Recall curve for all types of animals

7.5.4. F1 Curve:

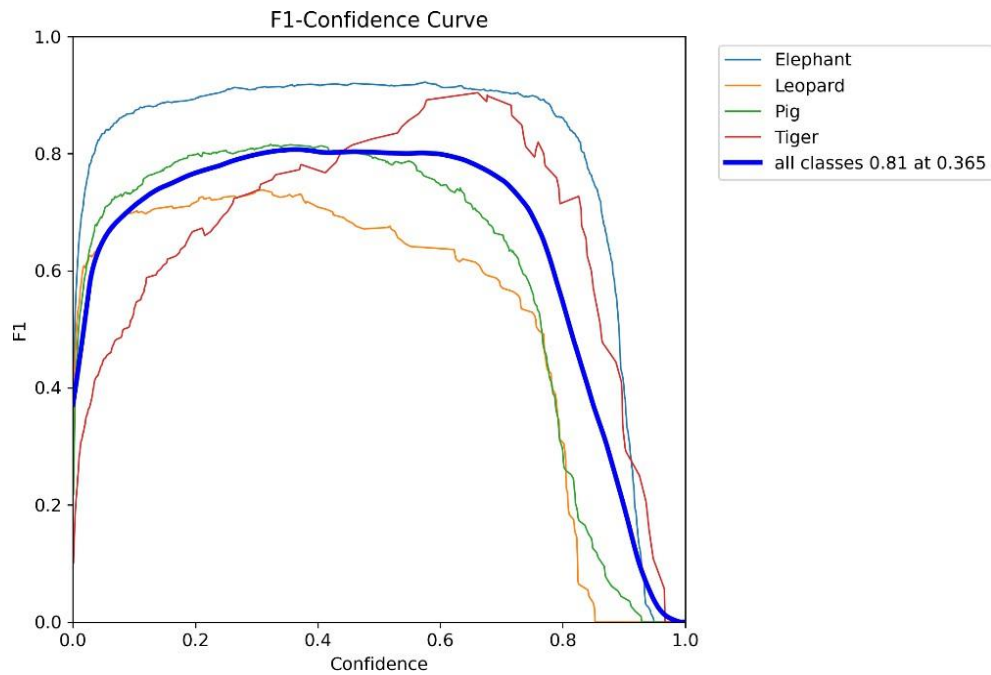


Figure:7.5.4 –F1 curve with Confidence for all types of animals

7.5.5.Labels Information chart:

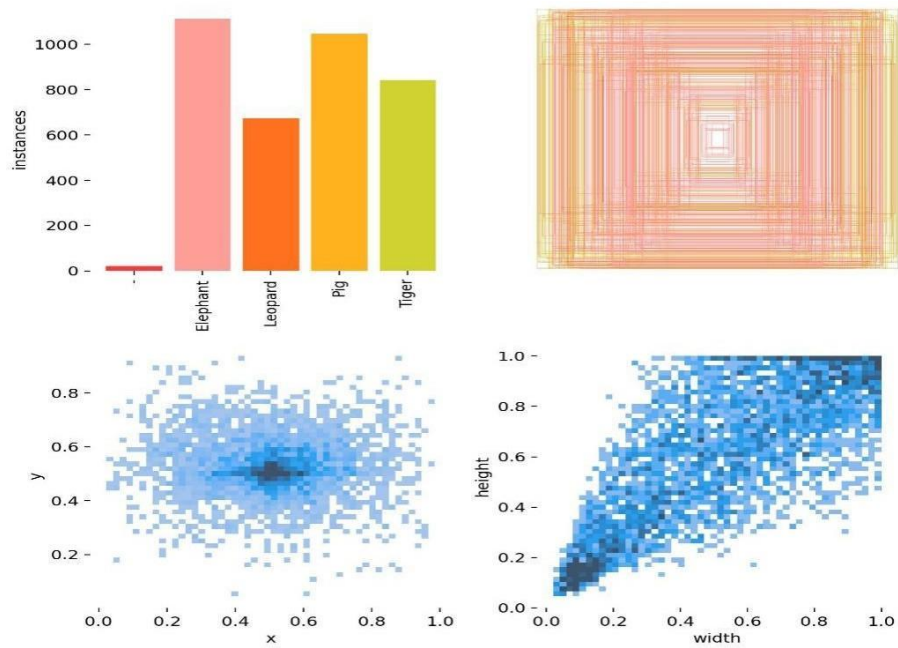


Figure:7.5.5 –Labels Information chart for all animals

7.5.6.Labels correlogram chart:

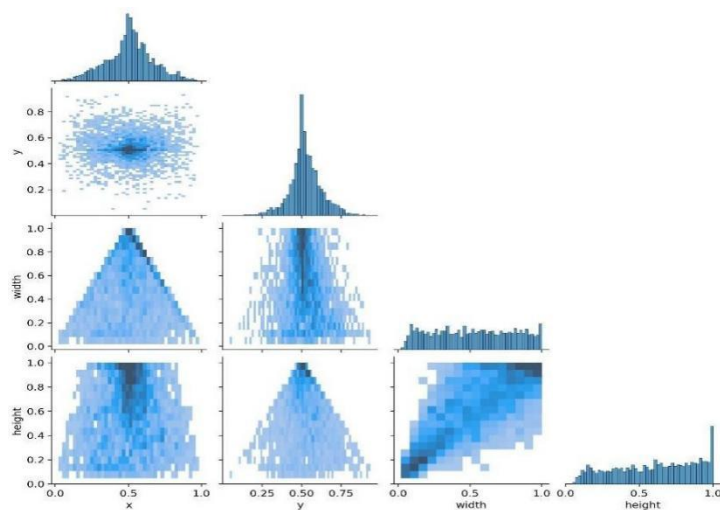


Figure:7.5.6 –Labels Correlogram

7.5.7. Charts showing the training, validation box loss with mAP values

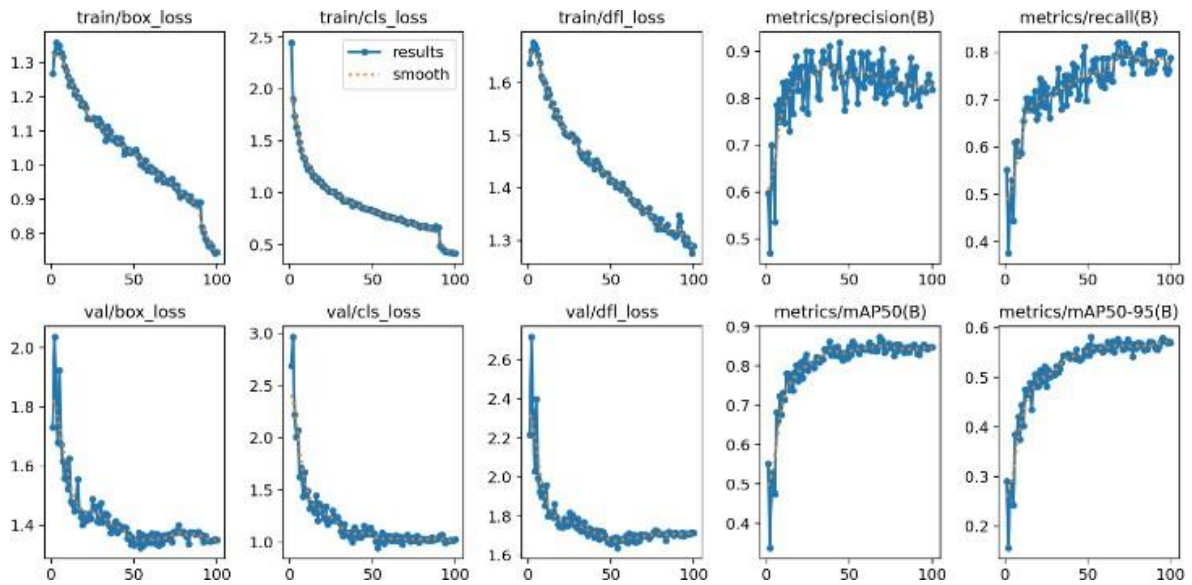


Figure:7.5.7 Performance Evaluation

7.5.8. Confusion matrix showing Predicted Vs True classes



Figure 7.5.8: Confusion Matrix

7.6 Model Output:

The model's output showcases a robust performance, processing 578 images and identifying 828 instances of wildlife species, including Elephant, Leopard, Pig, and Tiger classes. The overall precision stands at 89.8%, with a recall of 76.1%, reflecting a commendable balance between accurate detections and minimizing false positives. Notably, the model achieves an impressive mAP of 86.2%, underscoring its efficacy in object localization and classification. The efficiency metrics reveal swift processing times of 0.8ms for preprocessing, 4.7ms for inference, and 3.1ms for post-processing per image. Visual outputs, including confusion matrices and metrics, provide a comprehensive overview, further emphasizing the model's reliability in wildlife detection scenarios.

	Image pixels	mAP ^{val} 50-95	Speed CPU ONNX (ms)
Existing work on COCO dataset	640	44.9	1.2
Proposed work on Animal (custom dataset)	640	86.3	0.8

Table 7.6.1: Comparision of Model outcomes with Existing work

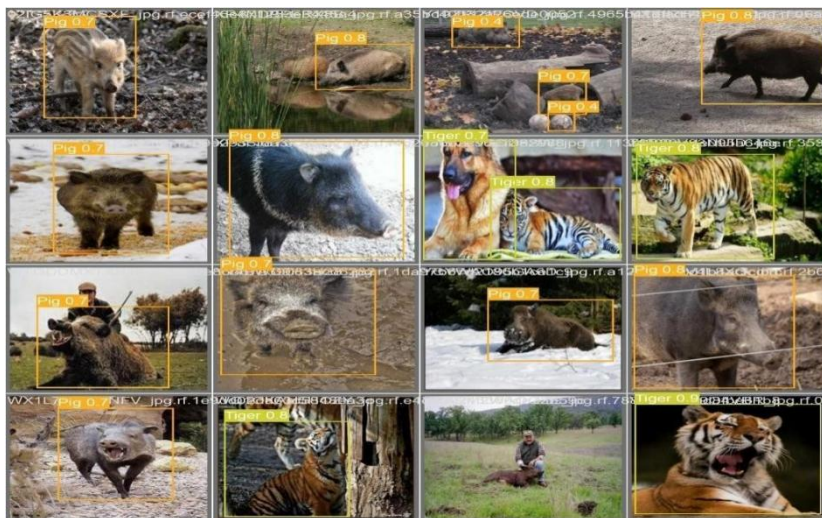


Figure 7.6.1 : Model prediction Results



Figure 7.6.2 :Elephant detected with 93% confidence



Figure 7.6.3 :Pig detected with 94% confidence



Figure 7.6.3 :Tiger detected with 95% confidence

8. CONCLUSION

The developed wildlife intrusion detection and deterrence system, employing deep learning techniques such as YOLOv8, has demonstrated significant efficacy in mitigating human-wildlife conflicts. The model exhibits robust object detection capabilities, accurately identifying various wildlife species, including Elephants, Leopards, Pigs, and Tigers, with a commendable precision of 89.8% and a mean Average Precision (mAP) of 86.2%. The system's successful implementation in real-world scenarios, as evidenced by the achieved results, highlights its potential to contribute substantially to wildlife conservation efforts.

Future Scope: In the future, there are several ways to improve and broaden the animal incursion detection system's capabilities. Integration with advanced sensor technologies, such as thermal imaging and acoustic sensors, could enhance detection accuracy, especially in challenging environmental conditions. Furthermore, by using reinforcement learning algorithms, the system could be able to adjust and perform better over time in response to input from the actual world. Collaborative efforts with wildlife conservation organizations and governmental agencies can facilitate the deployment of the system in diverse ecosystems, contributing to a broader and more effective conservation strategy. Furthermore, exploring edge computing solutions to deploy the model on edge devices could enable real-time, on-site decision-making, reducing dependence on centralized processing. Overall, these advancements promise to elevate the system's impact on wildlife conservation and human-wildlife conflict resolution.

9. REFERENCES

- [1] Patil, Hardiki, and Namrata Ansari. "Automated Wild-Animal Intrusion Detection and Repellent System Using Artificial Intelligence of Things." Proceedings of the 4th International Conference on Advances in Science & Technology (ICAST2021). 2021.
- [2] Saxena A., Gupta D.K., Singh S. "An Animal Detection and Collision Avoidance System Using Deep Learning." In: Hura G., Singh A., Siong Hoe L. (eds) Advances in Communication and Computational Technology, Springer,2020.
- [3] T. Fan, R. Sadeghian and S. Aram, "Deer-Vehicle Collisions Prevention using Deep Learning Techniques, "In Preceding of IEEE Cloud Summit, 2020.
- [4] Z. Zhao, P. Zheng, S. Xu and X. Wu, "Object Detection with Deep Learning: A Review," In Preceding of IEEE Transactions on Neural Networks and Learning Systems, 2019.
- [5] Amr Mohsen Jadi, "Implementing a Safe Travelling Technique to Avoid the Collision of Animals and Vehicles in Saudi Arabia" In Preceding of International Journal of Advanced Computer Science and Applications,2019
- [6] Sk. Nayab Rasool, T. SR.CH.Murthy, "Wildlife Monitoring in Zoological Parks Using RASPBERRYPI and Machine Learning" In preceding of International Journal of Recent Technology and Engineering (IJRTE),2019.
- [7] C. Bhagya and A. Shyna, "An Overview of Deep Learning Based Object Detection Techniques," In Preceding of 1st International Conference on Innovations in Information and Communication Technology (ICIICT), 2019.
- [8] N. Banupriya , S. Saranya, Rashmi Swaminathan, Sanchithaa Harikumar, Sukitha Palanisamy, "Animal Detection using Deep Learning Algorithm", In Journal of Critical Review, 2020.
- [9] Yu-Huei Cheng," A Development Architecture for the Intelligent Animal Care Management System Based on the Internet of Things and Artificial Intelligence", in Preceding of International Conference onArtificial Intelligence in Information and Communication(ICAIIC),2019.
- [10] Singh, M. Pietrasik, G. Natha, N. Ghouaiel, K. Brizel and N. Ray, "Animal Detection in Man-made Environments" In Preceding of IEEE Winter Conference on Applications of Computer Vision(WACV),2020.
- [11] S. Schneider, G. W. Taylor and S. Kremer, "Deep Learning Object Detection Methods for Ecological Camera Trap Data," In Preceding of 15th Conference on Computer and Robot Vision (CRV), 2018.
- [12] Mohan C. Nikam, Suraj Wagh, Priyanka Mahajan, Devayani Waghmare and Ketan Padewar, "Animal Welfare Monitoring Using IOT: A Review" In Preceding of International Journal for Modern Trends in Science and Technology,2018.
- [13] Verma, Gyanendra & Gupta, Pragya "Wild Animal Detection from Highly Cluttered Images Using Deep Convolutional Neural Network. "In preceding International Journal of Computational Intelligence and Applications, 2018

- [14] S. Sharma, D. Shah, R. Bhavsar, B. Jaiswal and K. Bamniya, "Automated Detection of Animals in Context to Indian Scenario," In Preceding of 5th International Conference on Intelligent Systems, Modelling and Simulation, 2014.
- [15] Anan Banharnsakun, Supanee Tanathong, "Object Detection Based on Template Matching through Use of Best- So-Far ABC", In Hindawi Publishing Corporation, Computational Intelligence and Neuroscience, 2014.
- [16] Sreenu, G., Saleem Durai, M.A. "Intelligent video surveillance: a review through deep learning techniques for crowd analysis." In J Big Data 6 Journal, 2019.
- [17] Black, Jake. "RAID: Real-time Animal Identification and Deterrence."
- [18] Adami, Davide, Mike O. Ojo, and Stefano Giordano. "Design, development and evaluation of an intelligent animal repelling system for crop protection based on embedded edge-AI." IEEE Access 9 (2021): 132125-132139.
- [19] HAAKSTAD, OLE HENRIK. Development of System for Deterrence of Animals using Artificial Intelligence. MS thesis. University of Agder, 2021.
- [20] Rangarajan, Krishnan. "Digital Borders: Design of an Animal Intrusion Detection System Based on Deep Learning." Computer Vision and Image Processing: 5th International Conference, CVIP 2020, Prayagraj, India, December 4-6, 2020, Revised Selected Papers. No. Part III. 2021.
- [21] Ravor, Prashanth C., T. S. B. Sudarshan, and Krishnan Rangarajan. "Digital Borders: Design of an Animal Intrusion Detection System Based on Deep Learning." International Conference on Computer Vision and Image Processing. Singapore: Springer Singapore, 2020.
- [22] Marcoň, Petr, et al. "A system using artificial intelligence to detect and scare bird flocks in the protection of ripening fruit." Sensors 21.12 (2021): 4244.
- [23] Jukan, Admela, Xavi Masip-Bruin, and Nina Amla. "Smart computing and sensing technologies for animal welfare: A systematic review." ACM Computing Surveys (CSUR) 50.1 (2017): 1-27.
- [24] Antoniou, Sophie. Vineyard Automated Pest Deterrence System. Diss. Worcester Polytechnic Institute, 2021.
- [25] Singh, Rajeev, and Sukhwinder Singh. "Smart border surveillance system using wireless sensor networks." International Journal of System Assurance Engineering and Management 13.Suppl 2 (2022): 880-894.
- [26] Jeevitha, S., and Venkatesh Kumar. "A review of animal intrusion detection system." Int. J. Eng. Res. Technol 9.5 (2020): 1219-1221.
- [27] Maheswari, P. Uma, and Anjali Rose Rajan. "Animal intrusion detection system using wireless sensor networks." International Journal of Advanced Research in Biology Engineering Science and Technology (IJARBEST) 2.10 (2016).
- [28] Xue, Wenling, Ting Jiang, and Jiong Shi. "Animal intrusion detection based on convolutional neural network." 2017 17th international symposium on communications and information technologies (ISCIT). IEEE, 2017.
- [29] Sabeenian, R., N. Deivanai, and B. Mythili. "Wild animals intrusion detection using deep learning techniques." Int. J. Pharm. Res 12.4 (2020): 1053-1058.





International Conference on Computational Innovations
and Emerging Trends (ICCIET-2K24)

organized by

Department of Computer Science Engineering
& Allied Branches

Bonam Venkata Chalamayya Engg. College, Odalarevu,
AP-INDIA



Certificate of Presentation

Presented to

Dr. Ch. Venkata Narayana

of

Professor, Lakireddy Bali Reddy College of Engineering

For presenting a paper titled "ANIMAL DETERRENCE USING COMPUTER VISION AND
RASPBERRY PI" during 4th - 5th April, 2024.

Dr. BSN Murthy
(Co-Convenor)

Dr. P Subba Rao
(Convenor)

Dr. Gunamani Jena
(Conference Chair)

Dr. Chandra Mouli VSA
(Principal-BVCE)



International Conference on Computational Innovations
and Emerging Trends (ICCIET-2K24)

organized by

Department of Computer Science Engineering
& Allied Branches

Bonam Venkata Chalamayya Engg. College, Odalarevu,
AP-INDIA



Certificate of Presentation

Presented to

Parsa Pallavi

of

Student, Lakireddy Bali Reddy College of Engineering

For presenting a paper titled "ANIMAL DETERRENCE USING COMPUTER VISION AND
RASPBERRY PI" during 4th - 5th April, 2024.

Dr. BSN Murthy
(Co-Convenor)

Dr. P Subba Rao
(Convenor)

Dr. Gunamani Jena
(Conference Chair)

Dr. Chandra Mouli VSA
(Principal-BVCE)



International Conference on Computational Innovations
and Emerging Trends (ICCIET-2K24)

organized by

Department of Computer Science Engineering
& Allied Branches

Bonam Venkata Chalamayya Engg. College, Odalarevu,
AP-INDIA



Certificate of Presentation

Presented to

Gundepalli Dedeepya

of

Student, Lakireddy Bali Reddy College of Engineering

For presenting a paper titled "ANIMAL DETERRENCE USING COMPUTER VISION AND
RASPBERRY PI" during 4th - 5th April, 2024.

Dr. BSN Murthy
(Co-Convenor)

Dr. P Subba Rao
(Convenor)

Dr. Gunamani Jena
(Conference Chair)

Dr. Chandra Mouli VSA
(Principal-BVCE)

